A TECHNICAL RESEARCH PROJECT

Smart Search

Design and Implementation of a Semantic Code Navigation Extension for Visual Studio Code

Nawfal Jalloul

Software Engineering · Personal Research & Development

Project Type
Personal Productivity Tool

Date
May 2026

Primary Technology
TypeScript · Python · VS Code API

AI Services
Voyage AI · OpenAI · Pinecone

Languages Supported
12 Programming Languages

Status
Completed & In Active Use

Table of Contents

Abstract

This paper documents the design, implementation, and engineering challenges of Smart Search — a Visual Studio Code extension built to solve a specific personal productivity problem: the inability to navigate large codebases using natural language when you know *what* a piece of code does but not what it is *called*. The system combines a function-level code chunker (supporting twelve programming languages), a hash-based incremental indexing engine, GPT-powered summarisation, Voyage AI semantic embeddings, and a Pinecone vector database to enable sub-second natural language queries that return not just the matching function but the exact line within it. The paper describes the full architecture, each component in detail, and fourteen distinct engineering challenges that arose during development, along with the solutions that were implemented. Topics include multi-language syntax parsing, cost-efficient incremental indexing through a two-level hashing strategy, cross-machine portability, multi-user isolation via git email namespacing, and the use of parallel API calls via Python's ThreadPoolExecutor to keep search latency below one second. The system is in active personal use and represents a practical example of embedding AI services into a developer tool without sacrificing performance, privacy, or reliability.

KEYWORDS: semantic code search, vector embeddings, VS Code extension, Voyage AI, Pinecone, incremental indexing, function-level chunking, developer tooling, Python HTTP server, ThreadPoolExecutor

1. Introduction and Motivation

1.1 The Problem with Keyword-Only Search

Every developer who has worked on a medium or large codebase has had some version of the same experience: you need to find a specific piece of logic, but you cannot remember what it is called. You know it exists because you either wrote it yourself months ago or saw it referenced somewhere in the code. You know roughly what it does — "it checks whether the user's subscription is still active" — but you have no idea whether the function is called `checkSubscription`, `isActive`, `validatePlan`, `getUserStatus`, or something else entirely.

The standard response is to reach for Ctrl+Shift+F in VS Code and try a few keywords. You search for "subscription," then "active," then "plan," then "billing." Each search either returns hundreds of irrelevant results or nothing at all. You end up manually browsing through controller files, service classes, and middleware handlers hoping to spot the right function. Depending on the size of the project and how well-named the functions are, this can take anywhere from two minutes to half an hour.

This is not a rare edge case — it is an everyday occurrence for anyone working on inherited code, revisiting old projects, or navigating a large team's repository where naming conventions are inconsistent. Keyword-based search is a powerful tool, but it has a fundamental limitation: it assumes you already know the vocabulary of the codebase you are searching. The moment that assumption breaks, the tool stops working.

1.2 Why I Built This

I built Smart Search primarily because I kept running into this exact problem in my own day-to-day work. My primary codebase at the time was a PHP Laravel project with several hundred files and a relatively large number of contributors, each with their own conventions for naming functions. Returning to it after a few weeks away felt like being dropped into an unfamiliar system every time.

I was aware that semantic search — searching by meaning rather than by exact text — had become practical with modern language models and vector databases. I had used tools like GitHub Copilot and ChatGPT to ask questions about code, and while they worked reasonably well for small snippets, neither of them could search across an

entire local workspace and return an exact file and line number. That was the specific capability I wanted: something that lived inside my editor, understood the full context of the project, and could answer "where is the authentication token validation?" with a clickable result rather than a conversation.

The result is this project. It is not a research prototype — it is a real tool that I use in actual development work. That context shapes every design decision: performance matters because I am using it dozens of times a day, cost matters because I am paying for the API calls myself, and correctness matters because incorrect results are worse than no results.

1.3 Project Goals and Scope

The system was designed with the following explicit goals:

- **Live inside VS Code.** The search interface should be a native panel in the editor, not a browser tab or a separate application. Opening a tool should be one keyboard shortcut away.
- **Support both normal and semantic search.** Keyword and regex search should still be available — they are faster for queries where you know the exact text. AI search should be an additional mode, not a replacement.
- **Never re-embed unchanged code.** Every time VS Code opens a project, it should not re-pay for embeddings that were already computed. Only changed functions should ever be re-processed.
- **Support multiple languages in one project.** Real projects mix languages. A full-stack application might have TypeScript on the frontend, PHP on the backend, and Python in scripts. The tool should index all of them.
- **Handle multiple users and multiple machines correctly.** If I work on the same project from two laptops, the tool should reuse my existing embeddings on the second machine. If a colleague works on the same project, their vectors should never interfere with mine.
- **Not require Python to be installed on every machine.** Some collaborators only use JavaScript. The backend should optionally run on a remote server so that not everyone needs a local Python environment.

2. Background and Technical Context

2.1 How Vector Embeddings Work

The core idea behind semantic search is that text can be converted into a list of numbers — a vector — in such a way that texts with similar meanings produce similar vectors. This conversion is done by a neural network that has been trained on large amounts of text. The network learns to position semantically related pieces of text close to each other in a high-dimensional numerical space.

A concrete example: after training, the vector for the sentence "check if the subscription is expired" will be positioned close to the vector for a PHP function that contains `if ($this->expires_at < now())`, even though no word from the sentence appears in the code. The model has learned the association between the concept of expiry checking and the code patterns that implement it.

In this project, Voyage AI's `voyage-code-2` model is used to produce these vectors. The model outputs a 1,536-dimensional floating-point vector for any piece of text — which means each function in the codebase is represented as a list of 1,536 numbers. This vector captures the "meaning" of the function in a way that supports comparison with natural language queries.

2.2 Vector Databases and Cosine Similarity

Comparing a query vector against thousands of stored function vectors requires a specialised database. Pinecone is used here because it is purpose-built for vector similarity search. Vectors are stored in Pinecone along with metadata (file name, line numbers, function name) and can be queried by providing a new vector and asking for the top-K most similar ones.

The similarity metric used is cosine similarity, which measures the angle between two vectors rather than their absolute distance. A cosine similarity of 1.0 means the vectors point in exactly the same direction (identical meaning). A score of 0.0 means the vectors are perpendicular (unrelated). In practice, scores above 0.6 with `voyage-code-2` indicate genuinely relevant matches for specific queries, while scores below 0.35 are usually noise.

Vectors in Pinecone are organised into namespaces, which are isolated partitions. This project uses namespaces to keep each user's vectors separate from other users', even when working on the same codebase — an important design choice described in detail in Section 6.8.

2.3 Why Existing Tools Fall Short

At the time of building this project, several semantic code search tools existed, including GitHub's code search with natural language support and tools like Sourcegraph. However, none of them met the specific requirements of this use case:

- **GitHub's semantic search** requires code to be hosted on GitHub and only works on publicly available repositories or paid enterprise plans. Local-only projects are not supported.
- **Sourcegraph** is a powerful tool for large teams but requires significant infrastructure to self-host and is not designed for individual developers working locally.
- **Copilot and ChatGPT** can answer questions about code when pasted into the chat, but they cannot search the entire workspace, return clickable file/line references, or maintain an indexed state between sessions.
- **VS Code's built-in search** is keyword/regex only and does not understand meaning.

The gap was a lightweight, local-first, semantically-aware search tool that integrates directly into VS Code, costs almost nothing to run after the initial index, and works on any private codebase without requiring remote hosting. This project is that tool.

3. System Architecture

3.1 Overview and Layer Design

The system is divided into two layers: a TypeScript VS Code extension that handles the editor integration, the user interface, file watching, and local hash management; and a Python HTTP server that handles all AI operations — summarisation with GPT, embedding with Voyage AI, and storage/retrieval with Pinecone. The two layers communicate over HTTP on localhost.

The reason for this split is practical. The VS Code extension must be written in TypeScript because that is what the VS Code Extension API requires. The AI service libraries (OpenAI, Voyage AI, Pinecone) all have mature, well-documented Python SDKs. Rather than working around incomplete JavaScript equivalents, it was simpler and more maintainable to put all AI work in Python. The separation also means the Python server can be deployed remotely if a collaborator does not have Python installed locally — the extension just points at a different URL.

Figure 1 — Full system architecture

3.2 VS Code Extension Layer

The extension activates automatically when VS Code opens a workspace. The entry point is `extension.ts`, whose `activate()` function registers eight behaviours: a status bar indicator, a backend health check, a full workspace index on startup, a file-save listener for incremental re-indexing, a file-delete listener, a file-rename listener, a manual re-index command, and the search panel command that opens the UI.

All indexing is non-blocking — it runs in the background so that VS Code opens and becomes usable immediately. The status bar at the bottom of the screen shows progress during indexing ("⟳ Smart Search: indexing 3/47...") and a confirmation when it finishes ("✓ Smart Search: 5 files updated").

The search panel is a VS Code webview — an embedded browser window running inside the editor. It loads an HTML/CSS/JavaScript frontend from the `frontend/` folder. Because the webview runs in a sandboxed environment and cannot make direct HTTP requests to the backend, all communication goes through a message-passing bridge: the frontend sends messages to the extension using `vscode.postMessage()`, and the extension relays those requests to the Python backend via `fetch()`.

The replace functionality uses VS Code's `WorkspaceEdit` API, which means all replacements are undoable with Ctrl+Z and respect the editor's normal undo history. For "Replace All" operations, replacements within each file are applied from bottom to top to avoid character position shifts as earlier text is modified.

3.3 Python Backend Layer

The backend is a deliberately simple Python HTTP server built on the standard library's `http.server.HTTPServer`. There is no web framework — no Flask, no FastAPI, no Django. This choice was made to minimise the install footprint: the only pip dependencies are `openai`, `pinecone`, `voyageai`, and `python-dotenv`. A developer can have the server running in under thirty seconds on a fresh machine.

The server handles four HTTP endpoints. `GET /health` returns a JSON liveness response and is polled by the extension on startup. `POST /search` handles both normal regex search and AI semantic search, differentiated by a `searchType` field in the request body. `POST /index` receives batches of code chunks to summarise, embed, and store. `POST /wipe` clears all vectors in a Pinecone namespace, used when the user requests a full re-index.

All endpoints return JSON with CORS headers, allowing the extension's webview to call them directly if needed in future configurations. The server binds to `localhost:8000` by default, but both the host and port can be overridden via environment variables. On the extension side, the URL is read from a VS Code configuration setting (`smartSearch.backendUrl`), making it trivial to point the extension at a remote server.

3.4 External Services

Three external services are used, all configured via API keys in the backend's `.env` file:

Table 1 — External services and their roles

SERVICE	MODEL	ROLE IN THE SYSTEM	COST
Voyage AI	voyage-code-2	Converts code chunks and search queries into 1,536-dimensional vectors	200M tokens/month free
OpenAI	gpt-4o-mini	Generates plain-English summaries of functions; pinpoints relevant lines within search results	~$0.00015 per 1K input tokens
Pinecone	Serverless	Stores all function vectors and returns the closest matches to a query vector	Free tier: 2 GB, 100K queries/month

4. The Indexing Pipeline

Indexing is the process of converting source code into searchable vectors. It is the most complex part of the system, involving multiple sequential and parallel steps. The pipeline runs in three scenarios: on VS Code startup (a full workspace scan), on every file save (a targeted single-file update), and on demand when the user runs the "Re-index Workspace" command (a complete wipe and rebuild).

4.1 Walking the Workspace

The indexer begins by recursively walking all files in the open workspace. Not all files are candidates for indexing. The walker skips directories that are unlikely to contain developer-written code: `.git`, `node_modules`, `vendor`, `dist`, `build`, `__pycache__`, `.venv`, `out`, `.next`, and `coverage`. It also skips the extension's own data folder (`.smart-search/`) to prevent it from inadvertently indexing its own hash files.

Files are filtered by extension. Only files with extensions associated with supported programming languages are indexed: `.py`, `.ts`, `.tsx`, `.js`, `.jsx`, `.java`, `.go`, `.rs`, `.c`, `.cpp`, `.cs`, `.rb`, `.php`, `.swift`, and `.kt`. Binary files, images, configuration files, and documentation are not indexed. Files larger than 500 KB are also skipped on the assumption that they are auto-generated, minified, or contain data rather than developer-written logic.

4.2 Two-Level Hash Strategy

The most important performance optimisation in the entire system is the two-level hash comparison that happens before any network call is made. The principle is simple: only code that has actually changed since the last indexing run should be re-embedded. Everything else should be skipped entirely.

The hash state is stored locally in `.smart-search/index.json` — a JSON file inside the project directory that is automatically added to `.gitignore`. This file stores MD5 hashes of every indexed function. No embeddings are stored locally; those live in Pinecone. The local file is just the tiny fingerprints needed to detect changes.

```
For each file found by the walker:

  LEVEL 1 — Whole-file hash (one MD5, one dictionary lookup)
  ──────────────────────────────────────────────────────────
  fileHash = MD5( entire file content )

  if fileHash == index.json["src/auth.php"]["fileHash"]:
      ⟶ skip this file entirely. No chunking. No API calls.
  else:
      ⟶ proceed to Level 2

  LEVEL 2 — Per-function hash (one MD5 per function)
  ──────────────────────────────────────────────────
  chunks = chunkFile( content, relativePath, language )
         = [ { name:"verifyToken", content:"function...", ... }, ... ]

  For each chunk:
    funcHash = MD5( normalizeCode( chunk.content ) )
               ↑ normalizeCode strips blank lines and trailing spaces
               so formatting-only changes don't trigger re-embedding

    if funcHash == index.json[file]["functions"][name]["hash"]:
        keep existing Pinecone vector — skip
    else:
        queue for summarise + embed + upsert

  For each name in index.json[file] that no longer exists in chunks:
      queue its Pinecone ID for deletion

  Result: only the changed delta goes to the backend.
  An untouched 200-function file: 1 hash lookup, 0 API calls.
```

Figure 2 — Two-level hash strategy for incremental indexing

The structure of `index.json` reflects this two-level design. The outer keys are relative file paths. The value for each file contains the whole-file hash (for the Level 1 check) and a dictionary of function entries, each with its own content hash and its Pinecone vector ID:

```
{
  "src/auth.php": {
    "fileHash": "a3f2c1d4",              // Level 1: hash of entire file
    "functions": {
      "verifyToken": {
        "hash":    "d4e2f1b8",           // Level 2: hash of this function's code
        "chunkId": "src/auth.php::verifyToken"  // ID in Pinecone
      },
      "hashPassword": {
        "hash":    "b8c3a2f1",
        "chunkId": "src/auth.php::hashPassword"
      }
    }
  }
}
```

All paths in this file are relative to the workspace root (for example, `src/auth.php` rather than `/Users/nawfal/projects/ecommerce/src/auth.php`). This is what makes the index portable: if the project folder is renamed or moved to a different directory, the stored hashes are still valid.

4.3 Summarisation and Embedding

Once the changed functions have been identified, they are sent to the Python backend in a single HTTP POST to `/index`. The request body contains the list of chunks to embed, a list of Pinecone IDs to delete (for functions that changed or were removed), and the namespace that scopes all operations to this user and project.

The backend first deletes the old vectors from Pinecone, then processes the new chunks in two stages. The first stage generates a plain-English summary for each chunk using `gpt-4o-mini`. The prompt asks GPT to produce one to four sentences describing what the function does, whether it reads or writes data, and whether it contains logging or error handling. This summary is the "semantic bridge" — it is what allows a natural language query to match code that uses completely different vocabulary. A function called `getProductById()` that runs a SQL SELECT query gets a summary like "Fetches a single product's details from the database by its ID using a prepared statement. Returns false if the product is not found."

In the second stage, the summary is combined with the function's name and code and sent to Voyage AI's `voyage-code-2` model. The combined text that actually gets embedded looks like this:

```
Function: getProductById
Fetches a single product's details from the database by its ID
using a prepared statement. Returns false if not found.

function getProductById($id) {
    $result = $this->db->query(
        'SELECT * FROM products WHERE id = ?', [$id]
    );
    if (!$result) { error_log("Product $id not found"); return false; }
    return $result[0];
}
```

The resulting 1,536-dimensional vector encodes both the natural language meaning (from the summary) and the code structure. This dual encoding is why the system achieves consistently high cosine similarity scores — typically 60–85% — compared to 35–55% when embedding raw code without a summary.

4.4 Storing Vectors in Pinecone

Vectors are stored in Pinecone using the `upsert` operation, which inserts a new record or overwrites an existing one if the same ID is present. This idempotency is important for the cross-machine scenario described later: re-indexing an already-indexed function simply replaces its vector, it does not create a duplicate.

Each record in Pinecone contains the vector itself, a unique string ID (in the form `relativePath::functionName`), and a metadata dictionary with the file path, function name, type, start and end line numbers, a preview of the source code (capped at 1,000 characters), and the GPT-generated summary (capped at 500 characters). The metadata is what gets returned in search results — the user sees the summary, filename, and line range without ever needing to read the vector itself.

5. The Search Pipeline

5.1 Normal Text and Regex Search

Normal search works as a high-speed regex-based grep. When the user types a query and selects the "Normal" mode, the extension sends a POST to `/search` with the query, the workspace path, and options for case sensitivity, whole-word matching, and regex mode. The Python backend compiles the pattern with Python's `re` module and walks the workspace directory tree using `os.walk`.

For each file, it opens it with UTF-8 encoding (with error-replacement for non-UTF-8 files) and applies the compiled pattern to each line using `finditer`. Each match is returned with its file path, line number, line content, and character offsets of every match on that line. These offsets are what the frontend uses to highlight the matched text in the results list. When the user clicks a result, the extension uses these same offsets to highlight the text in the editor.

The backend respects the same ignore folder list as the indexer, and the user can additionally filter results by file include/exclude glob patterns — for example, restricting a search to `*.php` files or excluding `test*` directories.

5.2 AI Semantic Search

AI search takes a fundamentally different path. The query is not matched against file content at all — instead, it is converted into a vector and compared against the stored function vectors in Pinecone. The steps are as follows:

```
User query: "where do we check if the user subscription has expired"
(enforced minimum: 20 characters — see Section 6.11)
         |
         ▼
POST /search { searchType:"ai", query, namespace, threshold:40, ... }
         |
         ▼
1. embedder.embed_text(query)
   Voyage AI, voyage-code-2, input_type="query"
   (different from "document" — asymmetric embedding optimised for retrieval)
   → query_vector: [ 0.21, -0.54, 0.87, ... ]  (1,536 floats)
         |
         ▼
2. pinecone_client.query_chunks(query_vector, namespace, top_k=10)
   Pinecone computes cosine similarity between the query vector
   and every stored function vector in this user's namespace.
   Returns the 10 most similar, with their scores and metadata.
         |
         ▼
3. Filter: keep only results where score >= threshold
   (threshold = user's slider value / 100, default 0.35)
   Sort: highest score first
         |
         ▼
4. Apply file filters (filesInclude / filesExclude glob patterns)
         |
         ▼
5. Line locator — all results processed in PARALLEL
   (see Section 6.13 for parallel execution detail)
   For each result:
     - Read function's lines from disk
     - Ask GPT: which line best answers the query?
     - Return { line: N, content: "exact line text" }
         |
         ▼
Return to extension: [{
    file, name, start_line, end_line,
    score: 0.82,
    summary: "Checks if subscription is currently active...",
    match_line: 28,
    match_content: "if ($this->expires_at < now()) {"
}]
```

Figure 3 — AI semantic search pipeline

5.3 Line-Level Result Pinpointing

One subtlety of function-level vector search is that it tells you *which function* is relevant but not *where inside that function* the answer lives. A matched function spanning lines 5 through 42 is not very useful if the specific logic the user asked about is on line 21 — they still need to scan 38 lines manually.

The line locator solves this. After Pinecone returns the matched functions, a second LLM call is made for each result. The function's lines are read from disk and presented to `gpt-4o-mini` as a numbered listing alongside the original search query. GPT is asked to return a single JSON object identifying the most relevant line number and its text content:

```
# Numbered listing sent to GPT (excerpt):
21: if ($this->expires_at < time()) {
22:     $this->status = 'expired';
23:     $this->save();
24:     return false;
25: }

# GPT response:
{ "line": 21, "content": "if ($this->expires_at < time()) {" }
```

The returned line number is clamped to the valid range `[start_line, end_line]` to guard against occasional out-of-range responses from the model. When the user clicks the result in the UI, VS Code opens the file and positions the cursor directly at `match_line` — not at the start of the function, but at the specific line that answers the query.

6. Engineering Challenges and Solutions

This section is the core of the paper. Each challenge described here is a real problem that was encountered during development and had to be consciously solved. They are presented roughly in the order they were encountered.

C-01 Parsing Multiple Language Syntaxes

The chunker's job is to split a source file into its individual functions and methods. For this to work correctly, it must know where each function starts and, more critically, where it ends. The challenge is that different programming languages use completely different syntactic mechanisms to delimit code blocks, and a single project might contain files in several languages.

Three fundamentally different block-delimiter systems had to be handled. The first — and most common — is the brace system used by PHP, TypeScript, JavaScript, Java, Go, Rust, C#, Swift, Kotlin, C, and C++. In these languages, a block starts with `{` and ends with the matching `}`. The word "matching" is the important part: braces can be nested, so a simple forward scan for the next `}` does not work. The depth of nesting must be tracked.

The second system is Python's indentation-based scoping. Python has no closing brace. A `def` block ends when a non-blank line appears at the same indentation level as the `def` keyword itself. A class method ends when the next method definition begins at the same indent, or when a line at the class's indent level appears. This requires recording the indentation of the opening line and scanning forward for the first non-blank line that is not further indented.

The third system is Ruby's keyword-based scoping. Ruby uses `end` to close blocks opened by `def`, `class`, `module`, `if`, `do`, and several others. Since the same `end` keyword closes all of them, a counter must track how many openers have been seen relative to closers.

Three separate algorithms were implemented in `chunker.ts`:

```
// Algorithm 1: brace counting for C-style languages
function findBraceEnd(lines: string[], startIdx: number): number {
  let depth = 0, started = false;
  for (let i = startIdx; i < lines.length; i++) {
    for (const ch of lines[i]) {
      if (ch === "{") { depth++; started = true; }
      else if (ch === "}" && started && --depth === 0) return i;
    }
  }
  return lines.length - 1;
}

// Algorithm 2: indentation tracking for Python
function findPythonEnd(lines: string[], startIdx: number): number {
  const headerIndent = lines[startIdx].match(/^(\s*)/)?.[1].length ?? 0;
  for (let i = startIdx + 1; i < lines.length; i++) {
    if (lines[i].trim() === "") continue;  // blank lines are valid inside a block
    const indent = lines[i].match(/^(\s*)/)?.[1].length ?? 0;
    if (indent <= headerIndent) return i - 1;
  }
  return lines.length - 1;
}

// Algorithm 3: keyword counting for Ruby
function findRubyEnd(lines: string[], startIdx: number): number {
  let depth = 1;
  const openers = /^\s*(?:def|class|module|do|if|unless|while|begin|case)\b/;
  const closer  = /^\s*end\b/;
  for (let i = startIdx + 1; i < lines.length; i++) {
    if (openers.test(lines[i])) depth++;
    if (closer.test(lines[i]) && --depth === 0) return i;
  }
  return lines.length - 1;
}
```

On top of the block-end detection, each language also needs its own regex patterns for recognising function declarations. TypeScript alone requires five separate patterns to cover standalone functions, arrow functions, function expressions assigned to variables, exported versions of all of the above, and class methods. PHP, Java, Go, Rust, and each other language have their own distinct function declaration syntax.

One additional problem that was discovered during testing: when class-level chunks are indexed, they contain the combined source of all their methods. A class vector therefore tends to score highly for almost any query about

functionality in that file, because it "averages" all its methods and ends up vaguely relevant to everything. This caused class results to consistently appear at the top of AI search results while pushing the genuinely relevant individual methods down the list. The fix was to filter out `type === "class"` chunks before sending them to the backend. Individual methods are already indexed separately, so class chunks are redundant and harmful.

If a file's language is not in the supported set, or if the chunker finds no function declarations in a supported-language file, the entire file is returned as a single chunk of type `"file"`. This fallback ensures every file gets some representation in Pinecone.

C-02 Avoiding Redundant Re-Embedding

This was arguably the most consequential engineering decision in the project. The naive implementation of indexing re-embeds every function every time VS Code opens. For a project with two hundred functions, this means two hundred GPT calls for summarisation, followed by one Voyage AI batch call, every single time the developer opens their editor. At current pricing, that is roughly $0.03 per startup. Small in absolute terms, but it also means waiting fifteen to thirty seconds every morning before the tool is usable. Neither the cost nor the delay is acceptable for a daily-use tool.

The solution is the two-level hash strategy described in Section 4.2. The key insight is that the vast majority of code does not change between sessions. If two hundred functions were indexed yesterday and three have been edited today, there is absolutely no reason to re-process the other one hundred and ninety-seven. The hash comparison identifies exactly which functions changed and ensures only those are re-processed.

The full comparison logic in `workspaceIndexer.ts` looks like this:

```
// For each file found by the walker:
const fileHash = hashContent(content);  // MD5 of entire file
const existing = localIndex[relativePath];

// Level 1: if the whole file is unchanged, skip it entirely
if (existing && existing.fileHash === fileHash) return false;

// Level 2: file changed — find which functions changed
const chunks = chunkFile(content, relativePath, language)
  .filter(c => c.type !== "class");  // exclude class aggregates

for (const chunk of chunks) {
  const funcHash = hashFunction(chunk.content);  // MD5 of normalised code
  const existingFunc = existingFunctions[chunk.name];

  if (!existingFunc || existingFunc.hash !== funcHash) {
    if (existingFunc) toDelete.push(existingFunc.chunkId);  // replace old vector
    toEmbed.push({ ...chunk, language });
  } else {
    newFunctions[chunk.name] = existingFunc;  // keep unchanged entry
  }
}
// Functions in the index that no longer exist in the file were deleted
for (const name of Object.keys(existingFunctions)) {
  if (!newChunkNames.has(name)) toDelete.push(existingFunctions[name].chunkId);
}
```

REAL-WORLD IMPACT

In a project with 200 functions where 3 were edited overnight: without hashing, 200 GPT calls + 200 Voyage AI tokens are consumed every startup. With hashing: 3 GPT calls + one small Voyage AI batch. The savings are 98.5% of API cost and roughly 97% of startup time on every run after the first.

C-03 Formatting-Insensitive Hashing

The two-level hash strategy works well until a code formatter enters the picture. Formatters like Prettier (JavaScript/TypeScript), Black (Python), and gofmt (Go) are commonly used in development workflows. They regularly reformat code by adding or removing blank lines, adjusting trailing whitespace, and changing

indentation style. None of these changes affect what the code does — they are purely cosmetic — but they would change the MD5 hash of every function they touched, triggering unnecessary re-embeddings.

Running Black on a one-hundred-function Python file after an unrelated change would cause all one hundred functions to be queued for re-embedding the next time VS Code opens, even though the logic in none of them changed. This would silently undermine the efficiency gains of the hash strategy.

The solution is a normalisation step applied before computing the hash used for comparison. The `normalizeCode()` function strips out the specific changes that formatters make:

```
function normalizeCode(content: string): string {
  return content
    .split("\n")
    .map(line => line.trimEnd())          // remove trailing spaces and tabs
    .filter(line => line.trim() !== "") // remove blank and whitespace-only lines
    .join("\n")
    .trim();
}
```

This normalisation means that adding or removing blank lines anywhere inside a function and adding or removing trailing whitespace on any line will produce the same hash. These are exactly the types of changes that formatters make. Meaningful changes — editing variable names, adding or removing logic, changing the function signature — still produce different hashes and correctly trigger re-embedding.

Critically, normalisation is applied *only for hash comparison*. The original, un-normalised code is what gets embedded and displayed in search results. The normalised version is discarded after the hash is computed.

C-04 Running Without Python Locally

The AI search features depend on a Python backend. This is straightforward when working on your own machine where Python is installed. It becomes a problem when sharing the tool with colleagues who primarily write JavaScript, or when deploying to a second machine that does not have Python set up, or — and this was actually encountered during development — on Windows machines where the Microsoft Store's "App Execution Aliases" intercept the `python` and `python3` commands before they reach the actual Python installation, causing a "Python was not found" error even when Python is installed and working.

The initial design of the extension hardcoded the backend URL as `http://localhost:8000`, making it impossible to point the extension at any other server. The fix was to move the URL to a VS Code configuration setting:

```
// config.ts — the only place the backend URL is read
export function getBackendUrl(): string {
  const config = vscode.workspace.getConfiguration("smartSearch");
  const url = config.get<string>("backendUrl", "http://localhost:8000");
  return url.replace(/\/$/, "");  // strip trailing slash for consistency
}
```

Every HTTP call in the extension goes through this function. There are no hardcoded URLs anywhere else. The default remains `localhost:8000`, so the tool works out of the box with no configuration for developers running the server locally. But anyone who sets `smartSearch.backendUrl` in their VS Code settings can point the extension at a remote machine running the Python server.

This opens up a practical deployment model for teams: one developer with Python installs and runs the server on their machine (or on a shared VM), and everyone else simply sets the URL in their VS Code settings without needing Python at all.

> **WINDOWS-SPECIFIC NOTE**
>
> On Windows, even after installing Python, the `python` command may be intercepted by a Microsoft Store alias. The fix: Settings → Apps → Advanced app settings → App execution aliases → disable the aliases for `python.exe` and `python3.exe`, then open a new terminal.

C-05 Handling Very Large Functions

Functions vary enormously in size. A simple helper might be five lines. A complex controller method in a PHP application can be several hundred lines. Embedding very large functions creates several interrelated problems.

The first is API token limits. Voyage AI's API accepts inputs up to a certain length. A three-hundred-line function can easily exceed this limit if sent verbatim. The second problem is embedding quality: a vector representing three hundred lines of mixed code is a vague average. If a user searches for "where do we log errors?", the vector for a

large function that mostly fetches database records but also logs an error somewhere in the middle will score poorly, because the logging behaviour is diluted by all the other code. The third problem is cost: charging for tokens is proportional to input length. Sending enormous functions wastes money proportionally.

The solution is a set of hard caps applied at each stage of the pipeline:

Table 2 — Character/token caps at each pipeline stage

STAGE	CAP	RATIONALE
chunker.ts (stored in Chunk object)	6,000 chars	Limits what is sent to the backend in the first place
summarizer.py (sent to GPT)	3,000 chars	Keeps summarisation prompt focused; reduces token cost
embedder.py (sent to Voyage AI)	8,000 chars	Voyage AI API input limit
Pinecone content metadata field	1,000 chars	Pinecone per-field metadata size limit
Pinecone summary metadata field	500 chars	Keeps result payloads compact

When a function exceeds 6,000 characters, only the first 6,000 are processed. This is a reasonable trade-off because the function's name, signature, docstring, and opening logic — which carry the most semantic signal for any retrieval task — almost always appear at the beginning. The error handling, edge cases, and return statements that dominate the long tail of a large function tend to be less discriminative for the kinds of queries developers ask.

C-06 Filtering Irrelevant Results

Pinecone's query API is designed to always return results. When asked for the top-10 most similar vectors, it will return ten results whether the query is excellent or completely unrelated to anything in the index. In a codebase where a query has only two genuinely relevant functions, the other eight results are just the closest available vectors — and they can be quite far away in semantic space.

Showing these weak results is counterproductive. A developer who sees eight irrelevant results alongside two good ones will quickly lose confidence in the tool. Worse, they might assume the relevant function does not exist and

start writing a duplicate. It is better to show two good results and nothing else than to show two good results buried in noise.

The solution is a configurable similarity threshold. After Pinecone returns results, the backend filters out any result whose cosine similarity score is below the threshold:

```
# server.py — filter and sort after Pinecone returns
results = sorted(
    [r for r in all_results if r["score"] >= threshold],
    key=lambda r: r["score"],
    reverse=True,
)
```

The threshold is a number between 0.0 and 1.0, derived by dividing the user's slider value (1–100) by 100. The default is 0.35, defined in `config.py` as `DEFAULT_AI_THRESHOLD`. If the user leaves the threshold field blank or enters an invalid value, this default is used. With Voyage AI's `voyage-code-2` model, scores above 0.60 indicate strong matches, scores in the 0.40–0.60 range are typically relevant but less precise, and anything below 0.35 is usually noise for specific queries.

C-07 Protecting the Index from Git

The `.smart-search/` folder inside the project directory contains two files: `project-id` (a UUID that identifies this project in Pinecone) and `index.json` (the hash store). Neither of these should ever be committed to version control. If they are pushed to a shared repository, several problems arise.

The most serious problem involves the project ID. If Developer A's `project-id` file is pushed and Developer B pulls it, they now share the same project ID. When Developer B indexes the project, they write vectors into the same Pinecone namespace as Developer A (since namespace = projectId :: userId) — or rather, into what would be the same namespace if they also shared the same user ID. Even if they do not share a namespace due to different user IDs, Developer B having the project-id file means their data is associated with A's project identity in ways that cause confusion. The `index.json` hashes are also machine-specific: they contain hashes computed from the code as it exists on A's machine, which might be different from the code on B's machine if B has local uncommitted changes.

The straightforward solution is to add `.smart-search/` to the project's `.gitignore`. However, relying on developers to remember to do this manually is unreliable. The extension handles it automatically: every time `getProjectId()` is called — which happens at the start of every indexing run — it calls `addToGitignore()`:

```
function addToGitignore(workspaceRoot: string): void {
  const entry = ".smart-search/";
  const gitignorePath = path.join(workspaceRoot, ".gitignore");

  if (fs.existsSync(gitignorePath)) {
    const existing = fs.readFileSync(gitignorePath, "utf8");
    if (!existing.includes(entry)) {  // only write if not already present
      fs.appendFileSync(gitignorePath,
        `\n# Smart Search — local function hashes\n${entry}\n`);
    }
  } else {
    fs.writeFileSync(gitignorePath,
      `# Smart Search — local function hashes\n${entry}\n`);
  }
}
```

The function checks whether the entry already exists before writing, so it is safe to call on every startup without accumulating duplicate lines. If no `.gitignore` exists, one is created.

C-08 Isolating Each User's Vectors

Pinecone is a shared cloud database. All users of the same Pinecone account share the same index. Without a proper isolation mechanism, Developer A and Developer B working on the same codebase would be writing vectors into the same storage space. The consequences range from subtle to serious: their index operations could overwrite each other's vectors (since vectors are identified by ID, and the IDs are derived from file paths and function names which would be identical for the same codebase), search results from one developer would be influenced by indexing decisions made by another, and debugging would become nearly impossible.

Pinecone supports namespaces — isolated partitions within an index that operate as completely separate vector spaces. The solution is to give every user-project combination its own namespace. The namespace is constructed from two components: a project identifier and a user identifier.

The project identifier is a UUID generated on the first run of the extension in a given project. It is stored in `.smart-search/project-id`. Because this file travels with the project folder (and is gitignored, so each developer generates their own on first use), the project ID is stable across restarts but unique per developer.

The user identifier is where the interesting design decision lies. It needs to satisfy several properties simultaneously: it must be stable across VS Code reinstalls, stable across different machines the same developer uses, different for different developers, and derivable without requiring the user to set up any accounts or configuration beyond what they would normally do for development work.

The solution is to use the developer's globally configured git email address, hashed with MD5. Every developer who uses git — which is effectively every software developer — has run `git config --global user.email` at some point. This value lives in `~/.gitconfig` on their machine and persists through VS Code reinstalls, system reinstalls, and any number of editor changes. Hashing it with MD5 produces a stable 32-character identifier without transmitting the raw email to any service:

```
// userId.ts
email = cp.execSync("git config --global user.email", { encoding: "utf8" }).trim();
return crypto.createHash("md5").update(email.toLowerCase()).digest("hex");
```

The `toLowerCase()` call ensures that `Nawfal@gmail.com` and `nawfal@gmail.com` produce the same hash, guarding against capitalisation inconsistencies in git configuration.

The resulting namespace format is `{projectId}::{userId}`. The double colon separator was chosen because it does not appear in either UUIDs (which use single hyphens) or MD5 hashes (which are hexadecimal), making the format unambiguous.

Table 3 — Namespace isolation behaviour across scenarios

SCENARIO	GIT EMAIL	NAMESPACE OUTCOME
Same developer, same machine, new VS Code session	Same	Same namespace — reuses existing vectors
Same developer, new laptop (same git email configured)	Same	Same namespace — reuses existing vectors
VS Code uninstalled and reinstalled	Same (in ~/.gitconfig)	Same namespace — nothing changes
Colleague with different git email, same codebase	Different	Different namespace — completely isolated
Colleague tries to edit original developer's vectors	Different email → different userId	Cannot access the original namespace by design

If the git email is not configured when the extension tries to index, the extension shows a clear error notification with the exact command to run (`git config --global user.email you@example.com`) and stops indexing. There is no silent fallback to a random ID — that would create a different namespace every session and would make the incremental hash strategy useless.

C-09 Working Across Multiple Machines

The namespace-based identity system described above handles the multi-user case elegantly. But what about a single developer who works on the same project from two different machines — a work laptop and a home laptop, for example?

On the second machine, the `.smart-search/` folder does not exist because it is gitignored. This means `index.json` is empty: the system has no record of which functions have been indexed before. From its perspective, every function in the project is new and needs to be embedded. This would trigger a full re-index — the same cost and time as the initial index on the first machine — even though the same vectors already exist in Pinecone.

This is somewhat unavoidable given the gitignore requirement: the local hash cache cannot be shared because it is deliberately excluded from version control. However, the situation is handled gracefully by the system's design rather than causing errors or corruption.

Because the developer uses the same git email on both machines, both machines derive the same userId and therefore the same Pinecone namespace. When the second machine re-indexes everything, it upserts vectors using the same IDs that the first machine already uploaded. Pinecone's upsert operation is idempotent: upserting a vector with an existing ID simply overwrites it with the new vector. No duplicates are created. The index remains consistent.

After the one-time full re-index on the second machine, `index.json` is populated with the current hashes. All subsequent sessions on that machine benefit from incremental indexing, just as on the first machine. The cost of working from a new machine is one additional full index. Everything after that is as efficient as on the original machine.

KEY INSIGHT

Same git email on any machine → same Pinecone namespace → existing vectors are reused or safely overwritten. No separate account management, no sync mechanism needed. The git identity that developers already have is sufficient.

C-10 Choosing the Right Embedding Model

The choice of embedding model has a direct and significant impact on search quality. OpenAI offers two general-purpose embedding models: `text-embedding-3-small` and `text-embedding-3-large`. Both are trained primarily on internet text — articles, Wikipedia pages, books, web content. They perform well on natural language similarity tasks but are not optimised for code.

Code is fundamentally different from natural language in ways that matter for embedding. Function and variable names are often abbreviated, camelCased, or invented compound words with no independent meaning outside the codebase (`pdResp` , `getUById` , `chkSubAct`). Punctuation carries semantic meaning (`{` , `->` , `=>` , `?>`) in a way it does not in English prose. The relationship between an English description and the code that implements it — "validate token expiry" mapping to `if ($token['exp'] < time())` — requires specifically learned associations that general text models do not have.

During early development, the system was tested with OpenAI embeddings. The scores for clearly relevant results were typically in the 35–55% range. At a threshold of 35%, too many weak results were included. Raising the threshold to 50% caused genuinely relevant functions to be filtered out. The signal-to-noise ratio was poor.

Voyage AI's `voyage-code-2` model was trained specifically on code and natural language pairs — GitHub code, documentation, technical writing, Stack Overflow questions and answers. It learns the specific associations between English descriptions and code implementations. With this model, clearly relevant results score in the 60–85% range, making it straightforward to set a threshold that captures genuine matches without noise.

A second important feature of Voyage AI is asymmetric embedding. Code chunks being stored use `input_type="document"`, which optimises the vector for retrieval. Search queries use `input_type="query"`, which shapes the query vector to point toward relevant documents rather than being self-similar. This asymmetry is what enables a natural language query to locate code that uses completely different vocabulary — the query and document vectors are designed to be complementary rather than identical in form.

Voyage AI also provides a free tier of 200 million tokens per month, which is more than sufficient for personal use of this tool.

C-11 Enforcing a Minimum Query Length

Very short queries produce poor results with semantic search, and this is not just a quality issue — it erodes user trust in the tool. A two-word query like "get user" embeds to a vector near the centre of the semantic space: it is somewhat similar to everything and precisely similar to nothing. Cosine similarity scores against such a query tend to cluster in the 30–45% range across almost all functions in the codebase, regardless of whether they are actually relevant. The results look and feel random.

The minimum query length of twenty characters was introduced after observing this behaviour. Twenty characters corresponds roughly to three or four meaningful words — enough to express a specific intent rather than a vague topic. The frontend checks this before sending the request and shows a warning if the query is too short: "AI search works best with a descriptive phrase of at least 4 words. Try describing what the code does."

Examples of the difference in practice:

- **"validate token"** (14 chars) — too vague; scores cluster across many functions
- **"validate user auth token"** (25 chars) — acceptable; returns relevant results

- **"check if the authentication token has expired and is still valid"** (65 chars) — ideal; very precise results

This restriction applies only to AI search. Normal keyword search has no minimum length — searching for a two-character variable name with regex mode is entirely valid.

C-12 Bridging English and Code with GPT Summaries

Even with a code-specialised model like `voyage-code-2`, embedding raw code alone leaves a semantic gap between natural language queries and the code that implements their intent. This gap is widest for codebases with terse naming conventions, domain-specific abbreviations, or functions whose names do not match the developer's vocabulary for the concept.

Consider a PHP function:

```
function getProductById($id) {
    $result = $this->db->query(
        'SELECT * FROM products WHERE id = ? AND deleted_at IS NULL',
        [$id]
    );
    if (!$result) {
        error_log("Product not found: $id");
        return false;
    }
    return $result[0];
}
```

A developer searching for "fetch a single product from the database by its ID" would expect this to be the top result. But the code itself contains very little of that vocabulary. The words "fetch," "single," and "database" appear nowhere. The code surface is dominated by PHP syntax — dollar signs, arrows, question marks, SQL keywords. Without help, the cosine similarity between the query vector and this function's raw code vector is around 42%, which is below the default threshold and would cause this result to be filtered out.

The GPT summarisation step bridges this gap. Before embedding, `gpt-4o-mini` produces a plain English description of the function's purpose. For the function above, it might generate: "Fetches a single product from the database by its ID using a prepared SELECT query. Returns the first matching row or false if the product is not found. Includes error logging on miss." This summary is prepended to the code before embedding:

```
// What actually gets sent to Voyage AI:
Function: getProductById
Fetches a single product from the database by its ID using a
prepared SELECT query. Returns false if not found. Includes
error logging on miss.

function getProductById($id) {
    $result = $this->db->query('SELECT * FROM products WHERE id = ?', [$id]);
    ...
```

With this enriched text, the cosine similarity against "fetch a single product from the database by its ID" rises to approximately 0.81 — well above the default threshold and genuinely ranking this result first.

The GPT prompt is carefully constructed to extract the three most useful dimensions for search: what the function does and why, whether it reads or writes data (SQL, API calls, file operations), and whether it contains logging or error handling. These three dimensions correspond directly to the most common types of developer searches: "where do we fetch X from the database?", "where do we call the payment API?", "where is the error handling for authentication?".

The summary is also stored in Pinecone metadata and displayed in search result cards. This gives the developer an immediate plain English description of the matched function before they click through to the code.

C-13 Parallelising API Calls

Two stages of the pipeline require one LLM call per item: function summarisation during indexing (one `gpt-4o-mini` call per new or changed function) and line location during search (one `gpt-4o-mini` call per matched result). If these calls were made sequentially, the latency would grow linearly with the number of items — making both indexing and search unacceptably slow at any reasonable scale.

A typical `gpt-4o-mini` call takes 300–500 milliseconds. Summarising ten functions sequentially would take 3,000–5,000 milliseconds — three to five seconds of latency just for the summarisation step, before any embedding or Pinecone operations. Running the line locator on eight search results sequentially would take 2,400–4,000 milliseconds, making the total AI search time four to five seconds. Neither of these is acceptable.

The solution is Python's `concurrent.futures.ThreadPoolExecutor`, which allows multiple functions to be submitted to a thread pool and executed concurrently. Since GPT API calls are I/O-bound — the Python code is

waiting for an HTTP response, not doing computation — Python's Global Interpreter Lock (GIL) does not limit concurrency. Network I/O releases the GIL, allowing all threads to genuinely run in parallel:

```
# server.py — parallel summarisation during indexing
with ThreadPoolExecutor(max_workers=max(1, len(chunks_to_embed))) as executor:
    futures = {
        executor.submit(summarize_chunk, chunk): chunk
        for chunk in chunks_to_embed  # all N calls submitted simultaneously
    }
    for future in as_completed(futures):
        chunk = futures[future]
        try:
            chunk["summary"] = future.result()
        except Exception as e:
            chunk["summary"] = ""  # summarisation failure is non-fatal

# same pattern for line location during search
with ThreadPoolExecutor(max_workers=max(1, len(results))) as executor:
    futures = {
        executor.submit(find_relevant_line, query, r, workspace_path): r
        for r in results
    }
```

The worker count is set to `max(1, len(items))` — one thread per item — so all API calls start at the same time. The total wall-clock time for N parallel calls equals roughly the time of the single slowest call, which is typically 400–500 milliseconds regardless of N.

Table 4 — Sequential vs parallel API call timing

OPERATION	ITEMS (N)	SEQUENTIAL TIME	PARALLEL TIME	SPEEDUP
Summarise functions (indexing)	10	~3,500 ms	~450 ms	7.8×
Line-locate results (search)	8	~2,800 ms	~400 ms	7.0×
Summarise functions (indexing)	3	~1,050 ms	~420 ms	2.5×
Line-locate results (search)	3	~1,050 ms	~380 ms	2.8×

The combination of parallel summarisation during indexing and parallel line location during search is what keeps the overall AI search response time below one second. Without parallelism, a search returning five results would take over two seconds just for line location — and that is before accounting for the initial embedding and Pinecone query.

7. A Complete Worked Example

To make the full system concrete, this section traces two complete scenarios end to end: the first time the extension runs on a new project, and a subsequent AI search.

Scenario A — First Run on a New Project

```
Developer opens /projects/ecommerce-api in VS Code for the first time.

extension.ts activate() fires:
  ├─ Creates status bar: "$(search) Smart Search"
  ├─ Pings GET /health → Python responds { ok: true }
  └─ Calls indexWorkspace() in background (non-blocking)

indexWorkspace():
  ├─ getProjectId():
  │    .smart-search/ doesn't exist → create it
  │    Generate UUID: "a3f2c1d4-e5b6-7890-abcd-ef1234567890"
  │    Write to .smart-search/project-id
  │    addToGitignore(): append ".smart-search/" to .gitignore
  ├─ getUserId():
  │    execSync("git config --global user.email")
  │    → "nawfal@gmail.com"
  │    MD5("nawfal@gmail.com") → "b7f2a1c5..."
  ├─ namespace = "a3f2c1d4-...::b7f2a1c5..."
  ├─ loadIndex() → {} (empty, first run)
  └─ walkFiles() → 47 files found

For each of 47 files:
  Status bar: "⟳ Smart Search: indexing 12/47"
  Level 1: no stored hash → proceed to Level 2
  chunkFile() → extract N functions
  hashFunction() for each → no stored hash → queue all

After walking all 47 files: 183 functions queued

POST /index {
  chunks:     [183 chunk objects],
  delete_ids: [],
  namespace:  "a3f2c1d4-...::b7f2a1c5..."
}

Python /index:
  ThreadPoolExecutor: 183 summarize_chunk() calls simultaneously
  → ~480ms total (vs ~64 seconds sequential)

  embed_chunks([183 chunks]):
  → ONE batched Voyage AI API call
  → 183 vectors returned, each 1,536 floats

  upsert_chunks(vectors, namespace):
  → 183 records written to Pinecone

Back in TypeScript:
  Update index.json with 183 function hashes
  saveIndex() → write .smart-search/index.json
```

```
    Status bar: "✓ Smart Search: 47 files updated"
    (resets to "$(search) Smart Search" after 5 seconds)

Total time: ~3.5 seconds
```

Figure 4 — First-run indexing trace for a 47-file project

Scenario B — AI Search Query

```
Developer opens the Smart Search panel (Command Palette → "Smart Search").

Webview loads:
  webviewManager reads frontend/index.html + styles.css + main.js
  → inlines CSS and JS into single HTML string
  → panel.webview.html = combined HTML

Developer selects "AI" mode, sets threshold to 40, types:
"where do we check if the user subscription has expired"
(45 characters — passes 20-char minimum check)

main.js posts message:
  vscode.postMessage({
    command: "search", searchType: "ai",
    query: "where do we check if the user subscription has expired",
    threshold: 40, filesInclude: "", filesExclude: ""
  })

extension.ts router → searchHandler.ts:
  postMessage({ command: "searchLoading" })  ← UI shows spinner
  namespace = "a3f2c1d4-...::b7f2a1c5..."

  POST /search {
    query:         "where do we check if...",
    searchType:    "ai",
    namespace:     "a3f2c1d4-...::b7f2a1c5...",
    threshold:     40,
    workspacePath: "/projects/ecommerce-api"
  }

Python /search (AI branch):
  threshold = 40 / 100 = 0.40

  Step 1: embed_text(query)
    Voyage AI, voyage-code-2, input_type="query"
    → query_vector: [0.21, -0.54, 0.87, ...] (1,536 floats)
    Time: ~95ms

  Step 2: pinecone.query(query_vector, namespace, top_k=10)
    Pinecone returns 10 closest:
      #1  score=0.84  SubscriptionService.php::checkActive
      #2  score=0.79  SubscriptionService.php::isExpired
      #3  score=0.61  AuthMiddleware.php::validateToken
      #4  score=0.42  Invoice.php::getDueDate
      #5  score=0.38  ← filtered out (below 0.40)
      ... (all remaining below 0.40 — filtered out)
    Time: ~50ms

  Step 3: 4 results remain above threshold
```

```
  Step 4: Line locator — ALL 4 IN PARALLEL:
    Thread 1: find_relevant_line(query, checkActive, workspace)
      reads SubscriptionService.php lines 12–45
      numbered listing: "28: if ($this->expires_at < now()) {"
      GPT: {"line": 28, "content": "if ($this->expires_at < now()) {"}
    Thread 2: find_relevant_line(query, isExpired, ...)  → line 67
    Thread 3: find_relevant_line(query, validateToken, ...) → line 34
    Thread 4: find_relevant_line(query, getDueDate, ...) → line 82
    All complete in ~390ms (parallel)

Return JSON:
  results: [
    { file:"src/billing/SubscriptionService.php",
      name:"checkActive", start_line:12, end_line:45,
      score:0.84, summary:"Checks if subscription is active...",
      match_line:28, match_content:"if ($this->expires_at < now()) {" },
    { ... isExpired, score:0.79, match_line:67, ... },
    { ... validateToken, score:0.61, match_line:34, ... },
    { ... getDueDate, score:0.42, match_line:82, ... }
  ],
  total: 4, time_ms: 587

Developer sees 4 result cards. Clicks result #1.
VS Code opens SubscriptionService.php, cursor at line 28.
Total time from click to editor position: 587ms
```

Figure 5 — Complete AI search trace

8. Performance and Cost Analysis

Performance was a first-class concern throughout the design of this system. A search tool that takes five seconds to return results or an indexer that costs a dollar per day would be unusable regardless of its search quality. The following tables summarise the expected performance characteristics.

Table 5 — Indexing performance across common scenarios

SCENARIO	FILES	FUNCTIONS PROCESSED	TIME	EST. API COST
First run (cold index, 47 files)	47	183	~3.5s	~$0.005
Subsequent VS Code open, no changes	47	0 (all skipped)	<100ms	$0.00
File save, 3 functions changed	1	3	~500ms	<$0.001
Re-index workspace command	47	183 (full rebuild)	~3.5s	~$0.005

Table 6 — Search performance by mode

MODE	OPERATION	TIME	NOTES
Normal	Full regex search, 47 files	50–300ms	Depends on disk I/O speed
AI	Query embedding (Voyage AI)	~95ms	Single synchronous call
AI	Pinecone vector query	~50ms	Serverless tier latency
AI	Line location (8 results, parallel)	~390ms	8 simultaneous gpt-4o-mini calls
AI	Total end-to-end search	500–900ms	Consistently under 1 second

Monthly cost for a developer who opens VS Code once per day with a 47-file project and makes twenty AI searches per day, with an average of three functions changed per day:

- Daily incremental indexing (3 changed functions): <$0.001 × 30 days = <$0.03/month
- Daily AI searches (20 searches × 8 results × 1 line-locate GPT call): ~$0.002/day × 30 = ~$0.06/month
- Total: approximately $0.09/month — less than ten cents

The Voyage AI free tier (200M tokens/month) covers all embedding operations many times over. The only meaningful cost is the OpenAI API calls for summarisation and line location, and those are minimal because of the parallel batching and the hash-based strategy that avoids re-summarising unchanged functions.

9. Future Work

The current system is functional and in active daily use, but several meaningful improvements have been identified through use that were out of scope for the initial implementation.

Sliding-Window Chunking for Large Functions

Currently, functions longer than 6,000 characters are truncated — only the first 6,000 characters are embedded. This means that logic appearing later in a very large function will never be matched by search queries. A more robust approach would be to split large functions into overlapping windows (e.g. 3,000-character windows with 500-character overlap) and index each window as a separate sub-chunk with a modified ID. This would allow any part of any function to be discoverable, regardless of where it appears in the function body.

Cross-Project Search

The current system scopes all searches to the open workspace. Developers who maintain multiple related projects — a set of microservices, for example — often want to search across all of them simultaneously. A "search all my projects" mode could query multiple Pinecone namespaces (one per project) and merge the results. The namespace design already supports this; the extension would need to maintain a registry of known project namespaces.

Smarter Namespace Portability

When a developer opens a project on a second machine, the absence of `index.json` forces a full re-index even though the vectors already exist in Pinecone. A possible optimisation is to store a lightweight copy of the index hashes in a machine-independent location — for example, as a metadata attachment on the Pinecone namespace itself, or in a user-specific cloud store. This would allow the second machine to skip re-indexing entirely for unchanged functions.

Embedding Model Upgrades

Voyage AI periodically releases improved models. Vectors produced by `voyage-code-2` cannot be meaningfully compared against vectors produced by a future `voyage-code-3`, because the two models define different vector spaces. The system currently has no mechanism for detecting that stored vectors are from an older model and triggering a re-index. Adding a model version tag to the Pinecone metadata would enable this: when the configured model changes, the system would automatically detect the mismatch and queue a full re-index.

Query History and Saved Results

A local history of recent queries (stored in VS Code's `globalState`) and the ability to bookmark specific search results would improve day-to-day usability. This is a frontend feature that requires no backend changes.

Streaming Search Results

Currently the UI displays a loading spinner until all line-location calls complete and the full result set is returned at once. A streaming mode that displays each result as its line-location call finishes would make the interface feel significantly more responsive, especially for queries that return many results with high-latency line-location calls.

Multi-Root Workspace Support

VS Code supports opening multiple project folders simultaneously in a multi-root workspace. The extension currently only indexes the first folder (`workspaceFolders[0]`). Supporting all folders in a multi-root workspace would require per-folder project IDs and coordinated indexing across all roots.

Additional Language Support

The current chunker supports twelve language families. Adding support for SQL stored procedures, Solidity smart contracts, Dart (Flutter), Scala, and others would broaden the tool's usefulness for developers working in those ecosystems. The chunker architecture is designed for extension — adding a new language requires implementing a single parsing function and adding a case to the dispatch switch statement.

10. Conclusion

This paper has described Smart Search: a VS Code extension that brings semantic, meaning-based code search to any local project without requiring remote code hosting, a team subscription, or significant infrastructure. The system was built to solve a specific, recurring personal pain point — navigating large codebases when you know what a function does but not what it is called — and it does so effectively.

The technical contribution is not a new AI model or a novel algorithm. It is a careful integration of existing services — Voyage AI's code-specialised embeddings, OpenAI's GPT for summarisation and line location, and Pinecone's vector database — into a system that is practical, fast, and cheap to run. The engineering work is in the surrounding infrastructure: the hash-based change detection that avoids redundant re-embedding, the three-algorithm language chunker that handles twelve programming languages, the git-email namespace system that isolates users without requiring any account management, and the ThreadPoolExecutor parallelism that keeps search latency below one second.

Each of the thirteen engineering challenges described in Section 6 arose from a real problem encountered in building and using the tool. They were not anticipated in advance — they emerged from the experience of actually running the system and discovering where it broke down or behaved poorly. The solutions are all targeted and minimal: they solve exactly the problem they were designed to solve without introducing unnecessary complexity.

The result is a tool that I use in my own development work every day. When I return to a codebase after weeks away and need to find "where we validate the user's payment method," I type that sentence into the search panel and get a specific file, a specific function, and a specific line number in under a second. That is the goal the project was built to achieve, and by that measure, it succeeds.

Smart Search · Technical Report · Nawfal Jalloul · May 2026

VS Code Extension API · Voyage AI voyage-code-2 · OpenAI gpt-4o-mini · Pinecone Serverless